

1. Write a python program that takes an integer input from the user and determines whether the number is positive,negative,or zero using conditional statements (if,elif,and else).

Aim:

To develop a python program that checks whether a given number entered by the user is positive,negative,or zero using conditional statements.

Description:

In this program, the user is prompted to enter an integer value.The program then evaluates the entered number using conditional statements.

- If the number is greater than 0, it prints "The number is positive."
- If the number is less than 0, it prints "The number is negative."
- If the number is equal to 0, it prints "The number is zero."

Program:

```
num = int(input("Enter a number: "))  
if num > 0:  
    print("The number is positive.")  
elif num < 0:  
    print("The number is negative.")  
else:  
    print("The number is zero.")
```

output:

```
PS D:\25M11MC030> & C:/Users/saira/AppData/Local/Python/pythoncore-3.14-64/python.exe d:/25M11MC030/number.py  
Enter a number: 5  
The number is positive.  
PS D:\25M11MC030> & C:/Users/saira/AppData/Local/Python/pythoncore-3.14-64/python.exe d:/25M11MC030/number.py  
Enter a number: -4  
The number is negative.  
PS D:\25M11MC030> & C:/Users/saira/AppData/Local/Python/pythoncore-3.14-64/python.exe d:/25M11MC030/number.py  
Enter a number: 0  
The number is zero.  
PS D:\25M11MC030> █
```

2. Write a Python program that calculates the factorial of a number using both for loop and while loop. The program should demonstrate the use of loop control statements such as pass, continue, break, and else.

Aim:

To write a Python script that computes the factorial of a number using both for and while loops, showcasing the working of loop control statements:

- Pass - does nothing (placeholder)
- Continue – skips the current iteration
- Break – stops the loop
- Else – executes when the loop completes normally (without break)

Description:

: The factorial of a number n (written as $n!$) is the product of all positive integers less than or equal to n .

For example:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

$$6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720$$

This program performs the following steps:

1. Accepts an integer from the user.
2. Calculates the factorial using both a for loop and a while loop.
3. Demonstrate the use of:
 - o Pass as a null operation.
 - o Continue to skip an iteration.
 - o Break to exit from the loop when input is invalid.
 - o Else to print the result when the loop completes successfully.

Program:

```
# Factorial using FOR loop
num = int(input("Enter a number: ")) factorial = 1
```

```
for i in range(1, num + 1):
    if i == 0:
        pass
    if i == 0:
        continue
    if i < 0:
        print("Factorial not defined for negative numbers.")
        break
    factorial *= i
else:
    print("Factorial of num using for loop is:" ,factorial)
# Factorial using WHILE loop
n = num
fact = 1
i = 1
while i <= n:
    if i == 0:
        pass
    if i < 0:
        print("Invalid input!")
        break
    if i == 0:
        continue
    fact *= i
    i += 1
else:
    print("Factorial of num using while loop is:"s,fact)
```

Output:

```
PS D:\25M11MC030> & C:/Users/saira/AppData/Local/Python/pythoncore-3.14-64/python.exe d:/25M11MC030/number.py
Enter a number: 8
Factorial of num using for loop is: 40320
Factorial of num using while loop is: 40320
PS D:\25M11MC030> & C:/Users/saira/AppData/Local/Python/pythoncore-3.14-64/python.exe d:/25M11MC030/number.py
Enter a number: 9
Factorial of num using for loop is: 362880
Factorial of num using while loop is: 362880
PS D:\25M11MC030> 
```

3. Write a Python program to perform various operations on lists (adding, removing, sorting, reversing) and tuples (accessing, elements, slicing, and unpacking).

Aim:

To demonstrate the use of basic list and tuple operations in Python such as adding, deleting, sorting, reversing (for lists), and accessing, slicing, and unpacking (for tuples).

Description:

In this program:

- A list of numbers is created and several operations are performed :
 - o Append(), to add an element
 - o Remove() to delete an element
 - o Sort() to arrange the list in ascending order
 - o Reverse() to reverse the list order
- A tuple of animals is used to show:
 - o Accessing elements using indexing
 - o Extracting a range using slicing
 - o Unpacking the tuple into individual variables

Program:

```
print("=== LIST OPERATIONS ===")
marks = [85, 72, 90, 66, 78]
print("Original list:", marks)
marks.append(95)
print("After adding 95:", marks)
marks.remove(66)
print("After removing 66:", marks)
marks.sort()
print("After sorting:", marks)
marks.reverse()
```

```

print("After reversing:", marks)
print("\n=== TUPLE OPERATIONS ===")
fruits = ("apple", "banana", "cherry", "mango", "orange")
print("Original tuple:", fruits)
print("First element:", fruits[0])
print("Last element:", fruits[-1])
print("Sliced tuple (1:4):", fruits[1:4])
(f1, f2, f3, f4, f5) = fruits
print("Unpacked values:")
print("f1 =", f1)
print("f2 =", f2)
print("f3 =", f3)
print("f4 =", f4)
print("f5 =", f5)

```

Output:

```

PS D:\25M11MC030> & C:/Users/saira/AppData/Local/Python/pythoncore-3.14-64/python.exe d:/25M11MC030/number.py
=== LIST OPERATIONS ===
Original list: [85, 72, 90, 66, 78]
After adding 95: [85, 72, 90, 66, 78, 95]
After removing 66: [85, 72, 90, 78, 95]
After sorting: [72, 78, 85, 90, 95]
After reversing: [95, 90, 85, 78, 72]

=== TUPLE OPERATIONS ===
Original tuple: ('apple', 'banana', 'cherry', 'mango', 'orange')
First element: apple
Last element: orange
Sliced tuple (1:4): ('banana', 'cherry', 'mango')
Unpacked values:
f1 = apple
f2 = banana
f3 = cherry
f4 = mango
f5 = orange
PS D:\25M11MC030> 

```

4.Develop a Python script that creates a dictionary to store and retrieve contact details such as names and phone numbers. The program should demonstrate adding, updating, deleting, and retrieving entries from the dictionary.

Aim:

To write a Python program that uses a dictionary for managing contact information, allowing basic operations such as add, update,delete, and retrieve.

Description:

In this program,a dictionary is used to store contact details where:

- Keys represent the contact names.
- Values represent their corresponding phone numbers.

The program performs the following operation

1. .Adding contacts – Inserts new name-number pairs into the dictionary.
2. Updating contacts – Modifies the phone number of an existing contact.
3. Deleting contacts – Removes a contact from the dictionary.
4. Retrieving contact – Displays the phone number of a specific contact

Program:

```
# Create an empty dictionary to store contacts
contacts = {}
contacts["venkat"] = "81424-51441"
contacts["prakash"] = "94939-35991"
contacts["manibabu"] = "93986-58642"
print("Contacts after adding:")
print(contacts)
contacts["venkat"] = "85559-79615"
print("\nContacts after updating venkat's number:")
print(contacts)
del contacts["manibabu"]
print("\nContacts after deleting manibabu:")
print(contacts)
```

```
print("\nPhone number for prakash:")  
print(contacts["prakash"])
```

output:

```
PS D:\25M11MC030> & C:/Users/saira/AppData/Local/Python/pythoncore-3.14-64/python.exe d:/25M11MC030/number.py  
Contacts after adding:  
{'venkat': '81424-51441', 'prakash': '94939-35991', 'manibabu': '93986-58642'}  
  
Contacts after updating venkat's number:  
{'venkat': '85559-79615', 'prakash': '94939-35991', 'manibabu': '93986-58642'}  
  
Contacts after deleting manibabu:  
{'venkat': '85559-79615', 'prakash': '94939-35991'}  
  
Phone number for prakash:  
94939-35991  
PS D:\25M11MC030> █
```


